



TRABAJO PRÁCTICO FINAL

Cada estudiante/equipo debe elegir UNA de las siguientes opciones y desarrollarla en Python, C o C#. Serán mejor valoradas aquellos trabajos que sean desarrollados en C# o C si su funcionamiento fuese acertado. Si la consigna lo especifica, se debe respetar el lenguaje solicitado.

La exposición del trabajo se realizará de forma dinámica (entre 5/10 minutos por equipo) donde deberán explicar brevemente lo desarrollado. Si ameritara, el docente podría realizar preguntas al respecto para verificar el entendimiento de lo expuesto. Su aprobación permite la promoción directa de la materia si cumpliera con los requisitos para dicho caso (parciales aprobados con notas no inferiores a 6 y con promedio de 7).

Entrega: individual o en grupos de hasta 2 integrantes. Se debe entregar:

- Código fuente
- Informe PDF con el formato ya practicado en el TPº1 incluyendo:
 - *Portada*
 - *Introducción*
 - *Objetivo del TP*
 - *Decisiones tomadas*
 - *Fundamentación con los contenidos vistos*
 - *Documentación del proceso: sistema operativo utilizado y versión, pruebas, errores, aciertos, código final y su funcionamiento. Deben incluir capturas de ello pero sabiendo seleccionar aquellas que aporten información visual a lo explicado*
 - *Conclusión*
- Link a un video (breve) cargado en alguna plataforma (Google Drive/YouTube) mostrando el correcto funcionamiento del programa (para tener respaldo de que verdaderamente funciona ante cualquier inconveniente durante la exposición) con nombre “grupoX_nombreDeEjercicio.mp4”.



Propuestas:

Opción 1 (Planificación de procesos)

Simulador de Planificador de Procesos

Simular cómo un sistema operativo decide qué proceso ejecutar a continuación.

Se debe implementar:

- Definir una clase PCB con: PID, nombre, estado (READY / EJECUTANDO / ETC). Es decir que debe incluir los elementos (fundamentales) que componen al PCB.
- Implementar una cola de procesos READY/PREPARADO.
- Implementar al menos 2 algoritmos: FCFS y uno a elección (SJF, Round Robin o por Prioridad).
- Al finalizar la simulación, mostrar el orden en que fueron ejecutados los procesos bajo cada algoritmo y comparar los resultados. Por ejemplo: *FCFS* ejecutó → *P1, P3, P2* / *SJF* ejecutó → *P2, P1, P3*.

Opción 2 (Hilos)

Productor y Consumidor con Hilos

Simular el problema clásico de productor/consumidor usando hilos y un buffer compartido.

Se debe implementar:

- Crear un hilo Productor que genere ítems y los coloque en un buffer de capacidad limitada.
- Crear un hilo Consumidor que tome ítems del buffer y los procese (imprimirlos o acumularlos).
- Usar un mecanismo de sincronización (mutex o semáforo) para evitar condiciones de carrera.

Opción 3 (Procesos)

Monitor de Procesos del Sistema

*Construir una herramienta que inspeccione y muestre información sobre los procesos reales del sistema operativo. Pueden utilizar *psutil* en Py.*

Se debe implementar:

- Listar los procesos activos con su PID, nombre y uso de CPU/memoria.
- Permitir buscar un proceso por nombre o PID. Dado un proceso, mostrar su árbol de procesos: el proceso padre (PPID) y todos sus procesos hijos, indicando la jerarquía visualmente en consola
- Permitir terminar un proceso por PID, verificando primero que existe y pidiendo confirmación al usuario antes de hacerlo



Opción 4 (Archivos)

Sistema de Archivos por Consola

Implementar una mini shell que permita operar sobre archivos y directorios del sistema.

Se debe implementar:

- Comandos mínimos: listar archivos, copiar, mover y borrar
- Verificar en cada operación que el origen existe y si corresponde, que el destino no
- Mostrar mensajes de error claros cuando una operación falla
- Registrar cada operación en un archivo de log con fecha y hora

Opción 5 (IPC)

Chat entre procesos con Sockets

Implementar una comunicación entre dos procesos independientes usando sockets TCP.

Se debe implementar:

- Crear un proceso Servidor que escuche conexiones en un puerto local
- Crear un proceso Cliente que se conecte al servidor y permita enviar mensajes desde la consola
- El servidor debe recibir cada mensaje y mostrarlo en pantalla con el timestamp de llegada.
- Manejar correctamente la desconexión del cliente (el servidor no debe caerse)

Opción 6 (IPC)

Cola de Mensajes entre Procesos

Simular la comunicación asincrónica entre procesos mediante una cola de mensajes compartida.

Se debe implementar:

- Crear al menos 2 procesos (pueden ser hilos simulando procesos) que se comuniquen a través de una cola
- Un proceso **EMISOR** lee mensajes del teclado y los pone en la cola; un **RECEPTOR** los lee e imprime
- Mostrar el estado de la cola (cantidad de mensajes pendientes) cada vez que cambia



Opción 7 (IPC)

Shared memory

Cree 2 aplicaciones distintas en lenguaje C, en diferentes instancias de VS Code sobre Linux

Se debe implementar:

- Ambas aplicaciones deben poder acceder a un bloque de memoria compartida para permitir que una de ellas tome lo que se escribe por teclado, mientras que la segunda lo escribe en pantalla, limpiando dicha memoria
- Analice las complicaciones que surgen al intentar comunicar esos procesos. Explore las soluciones disponibles
- En base a lo realizado, en lugar de usar memoria compartida, cree un archivo y utilícelo para intercambiar la información entre las diferentes Aplicaciones

Opción 8 (IPC)

Comunicación entre Procesos con Pipes

Implementar la comunicación bidireccional entre dos procesos usando pipes (tuberías) en C#.

Se debe implementar:

- Crear dos programas independientes: un **ESCRITOR** y un **LECTOR**.
- El Escritor lee mensajes ingresados por consola y los envía a través de un pipe con nombre (named pipe).
- El Lector permanece a la escucha, recibe cada mensaje y lo muestra en pantalla.
- Manejar correctamente los casos de cierre: si el *Escritor* termina, el *Lector* debe detectarlo y cerrar sin errores.